UTS

PEMBELAJARAN MESIN



NAMA : Benedict Samuel Ardy

NIM : 202431149

KELAS : F

DOSEN : Dr. Dra. Dwina Kuswardani, M.Kom

NO.PC : 12

ASISTEN : 1. Jonathan Juan Setyanto

2. Muhammad Caisar Gemilang Pratama

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

BAB I PENDAHULUAN 3
1.1 Rumusan Masalah 3
1.2 Tujuan Masalah 3
1.3 Manfaat Masalah 3
BAB II LANDASAN TEORI 4
2.1 Prediksi Penyakit Jantung dengan Pembelajaran Mesin 4
2.2 Exploratory Data Analysis (EDA) 4
2.3 Preprocessing Data 4
2.4 K-Nearest Neighbors (KNN) 5
2.5 Decision Tree 5
2.6 Evaluasi Model Klasifikasi 5
BAB III HASIL 6
3.1 Deskripsi Dataset dan Lingkungan Kerja 6
3.2 Cell 1 – Import Library 6
3.3 Cell 2 – EDA: Memuat dan Memeriksa Dataset 6
3.4 Cell 3 – EDA: Visualisasi Target dan Korelasi 8
3.5 Cell 4 – Preprocessing dan Split Data 9
3.6 Cell 5 – Standardisasi Fitur 10
3.7 Cell 6 – Pemodelan KNN (K = 7) 10
3.8 Cell 7 – Pencarian Nilai K Optimal 11
3.9 Cell 8 – Pemodelan Decision Tree 12
3.10 Perbandingan Model dan Analisis 14
BAB IV PENUTUP 15
4.1 Kesimpulan 15
4.2 Saran 15
DAFTAR PUSTAKA 16

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

Praktikum UTS ini menggunakan dataset Heart Disease (01_heart_disease.csv) yang berisi 918 data pasien dengan 11 fitur klinis dan satu variabel target HeartDisease. Berdasarkan hasil pengerjaan, rumusan masalahnya adalah sebagai berikut.

1. Bagaimana karakteristik dataset Heart Disease berdasarkan hasil Exploratory Data Analysis (EDA), meliputi struktur data, statistik deskriptif, distribusi kelas target, dan korelasi antarfitur numerik?
2. Bagaimana tahapan preprocessing (one-hot encoding, pemisahan fitur dan target, pembagian data latih–uji, serta standardisasi) menyiapkan data agar dapat dilatih oleh model?
3. Bagaimana kinerja algoritma K-Nearest Neighbors (KNN) dan Decision Tree dalam memprediksi penyakit jantung, serta bagaimana pengaruh nilai K terhadap akurasi KNN?
4. Model manakah yang menghasilkan akurasi terbaik pada data uji dan faktor apa yang menyebabkan perbedaan kinerja kedua model?

1.2 Tujuan Masalah

Tujuan dan hasil yang diperoleh dari praktikum UTS ini adalah:

1. Memahami dataset melalui EDA, yaitu 918 baris dan 12 kolom tanpa nilai kosong, dengan kelas target relatif seimbang (508 pasien positif dan 410 negatif).
2. Menerapkan preprocessing sehingga diperoleh 15 fitur numerik hasil one-hot encoding, dengan pembagian 688 data latih dan 230 data uji.
3. Melatih dan mengevaluasi model KNN (K = 7) yang menghasilkan akurasi 0,8870 dan model Decision Tree yang menghasilkan akurasi 0,8043.
4. Membandingkan kedua model berdasarkan akurasi, precision, recall, F1-score, dan confusion matrix untuk menentukan model yang lebih sesuai.

1.3 Manfaat Masalah

Manfaat yang diperoleh dari praktikum UTS ini adalah (1) mahasiswa memahami alur lengkap proyek klasifikasi mulai dari EDA, preprocessing, pemodelan, hingga evaluasi; (2) mahasiswa memahami bahwa algoritma berbasis jarak seperti KNN membutuhkan standardisasi, sedangkan Decision Tree tidak; (3) hasil perbandingan model dapat menjadi gambaran awal pemanfaatan pembelajaran mesin sebagai alat bantu skrining risiko penyakit jantung, dengan catatan bahwa hasilnya bukan pengganti diagnosis tenaga medis.

BAB II
LANDASAN TEORI

2.1 Prediksi Penyakit Jantung dengan Pembelajaran Mesin

Penyakit jantung merupakan salah satu penyebab kematian terbesar di dunia sehingga deteksi dini sangat diperlukan agar penanganan medis dapat dilakukan tepat waktu (Majumder et al., 2023; Harikiran et al., 2024). Pembelajaran mesin terawasi (supervised learning) memanfaatkan data pasien yang sudah berlabel, misalnya usia, jenis kelamin, tekanan darah, dan kadar kolesterol, untuk melatih model yang memprediksi ada tidaknya penyakit jantung. Alur kerja yang lazim terdiri atas pengumpulan data, preprocessing untuk membentuk dataset yang bersih, penerapan algoritma klasifikasi, dan evaluasi kinerja model (Mary et al., 2022). Pada praktikum ini permasalahan dirumuskan sebagai klasifikasi biner, yaitu HeartDisease bernilai 0 (tidak) atau 1 (ya).

2.2 Exploratory Data Analysis (EDA)

EDA adalah tahap awal untuk memahami struktur, kualitas, dan pola data sebelum pemodelan. Statistik deskriptif (jumlah data, rata-rata, simpangan baku, nilai minimum, kuartil, dan maksimum) membantu mendeteksi sebaran dan nilai ekstrem pada fitur numerik. Enihe et al. (2025) menggunakan matriks korelasi dan penghitungan jumlah kelas target pada tahap EDA, dan dari situ ditemukan ketidakseimbangan kelas yang dapat memengaruhi kinerja model. Pemeriksaan distribusi target penting karena akurasi dapat menyesatkan ketika proporsi kelas timpang (Enihe et al., 2025). Adapun korelasi Pearson (r) mengukur kekuatan dan arah hubungan linear antara dua variabel numerik dengan rentang -1 sampai $+1$; nilai mendekati ± 1 menunjukkan hubungan kuat, sedangkan mendekati 0 menunjukkan hubungan lemah.

2.3 Preprocessing Data

Encoding variabel kategorikal. Sebagian besar algoritma pembelajaran mesin dirancang untuk menerima masukan numerik, sehingga variabel kategorikal harus dikodekan menjadi angka sebelum pelatihan (Pargent et al., 2022). One-hot encoding mengubah setiap kategori menjadi kolom biner (0/1). Opsi drop_first menghapus satu kategori pada setiap variabel untuk menghindari redundansi informasi antarkolom.

Pembagian data latih dan data uji. Data dipisah agar kinerja model diukur pada data yang tidak digunakan saat pelatihan. Enihe et al. (2025) memakai rasio 80:20 setelah membandingkannya dengan beberapa rasio lain; pada praktikum ini digunakan rasio 75:25 dengan random_state = 42 agar hasil dapat direproduksi.

Standardisasi fitur. StandardScaler mengubah setiap fitur sehingga memiliki rata-rata 0 dan simpangan baku 1 dengan rumus $z = (x - \mu) / \sigma$. Majumder et al. (2023) menerapkan StandardScaler untuk menjaga kestabilan pelatihan model dan mengurangi dampak outlier, sedangkan Harikiran et al. (2024) melakukan scaling pada data sebelum melatih kNN dan classifier lainnya. Nilai μ dan σ

dihitung hanya dari data latih (fit), kemudian diterapkan pada data uji (transform) agar informasi data uji tidak bocor ke proses pelatihan.

2.4 K-Nearest Neighbors (KNN)

KNN adalah algoritma pembelajaran berbasis instance yang mengklasifikasikan data baru menurut kelas mayoritas dari K tetangga terdekatnya di ruang fitur (Enihe et al., 2025). Kedekatan umumnya diukur dengan jarak Euclidean, $d(x, y) = \sqrt{\Sigma(x_i - y_i)^2}$ (Fatah & Aida, 2026). Karena bekerja berdasarkan jarak, fitur yang berskala besar (misalnya Cholesterol) akan mendominasi perhitungan apabila tidak distandardisasi. Takcı (2023) menunjukkan bahwa jumlah tetangga, metode pembobotan, dan ukuran jarak sangat memengaruhi akurasi KNN pada prediksi penyakit jantung. Secara umum, K yang kecil membuat model sensitif terhadap noise, sedangkan K yang besar menghasilkan batas keputusan yang lebih halus. Kelemahan KNN adalah proses prediksi yang lambat pada data besar karena jarak harus dihitung untuk setiap data uji (Enihe et al., 2025). Harikiran et al. (2024) melaporkan bahwa kNN dan Naive Bayes lebih unggul daripada Logistic Regression pada prediksi penyakit jantung.

2.5 Decision Tree

Decision Tree membagi data secara rekursif berdasarkan atribut yang memberikan pemisahan terbaik, diukur dengan information gain atau entropi pada J48 dan Gini index pada CART (Mary et al., 2022). Hasilnya berupa struktur pohon yang mudah diinterpretasi, tetapi model ini rentan mengalami overfitting (Enihe et al., 2025). Alkattan et al. (2025) menunjukkan bahwa pohon yang dangkal bersifat transparan namun lemah dalam klasifikasi, sedangkan pohon yang dalam menangkap interaksi antarfitur tetapi overfit; pemangkasan (pruning) dan pembatasan kedalaman digunakan untuk menyeimbangkan keduanya. Karena pemisahan pohon didasarkan pada ambang nilai suatu fitur dan bukan pada jarak, Decision Tree tidak memerlukan standardisasi fitur.

2.6 Evaluasi Model Klasifikasi

Kinerja klasifikasi diringkas dalam confusion matrix yang memuat True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN). Dari matriks tersebut dihitung metrik berikut (Mary et al., 2022; Enihe et al., 2025):

- Accuracy = (TP + TN) / (TP + TN + FP + FN)
- Precision = TP / (TP + FP)
- Recall = TP / (TP + FN)
- F1-score = 2 × Precision × Recall / (Precision + Recall)

Recall menjadi penting pada diagnosis penyakit karena biaya kesalahan negatif palsu (pasien sakit yang diprediksi sehat) umumnya lebih besar, sedangkan akurasi dapat menyesatkan jika kelas tidak seimbang (Enihe et al., 2025). F1-score merupakan rata-rata harmonik precision dan recall sehingga berguna untuk melihat keseimbangan keduanya (Mary et al., 2022).

BAB III
HASIL

3.1 Deskripsi Dataset dan Lingkungan Kerja

Praktikum dikerjakan dengan Python di Jupyter Notebook menggunakan pustaka NumPy, Pandas, Matplotlib, Seaborn, dan scikit-learn. Dataset yang digunakan adalah 01_heart_disease.csv yang terdiri atas 918 baris dan 12 kolom: 11 fitur (Age, Sex, ChestPainType, RestingBP, Cholesterol, FastingBS, RestingECG, MaxHR, ExerciseAngina, Oldpeak, ST_Slope) dan 1 target (HeartDisease). Seluruh cell kode diberi watermark NIM berupa komentar "# NIM: 202431149" pada baris pertama cell.

3.2 Cell 1 – Import Library

Kode

Import Library ¶

```
# NIM: 202431149
# Nama: Benedict Samuel Ardy
# Import Library yang dibutuhkan untuk analisis dan machine learning
import numpy as np           # untuk operasi array dan numerik
import pandas as pd          # untuk manipulasi dan membaca dataset
import matplotlib.pyplot as plt # untuk visualisasi grafik
import seaborn as sns        # untuk visualisasi statistik yang lebih menarik

# Mengatur agar plot muncul langsung di Jupyter Notebook
%matplotlib inline
```

Penjelasan

Cell ini mengimpor pustaka yang dibutuhkan: NumPy untuk operasi array, Pandas untuk membaca dan memanipulasi dataset, Matplotlib dan Seaborn untuk visualisasi, serta magic command %matplotlib inline agar grafik tampil langsung di dalam notebook. Cell ini tidak menghasilkan output.

3.3 Cell 2 – EDA: Memuat dan Memeriksa Dataset

Kode

EDA

```
# NIM: 202431149
# Memuat dataset Heart Disease
df = pd.read_csv('01_heart_disease.csv', sep=",")

# Menampilkan 5 baris pertama data
print("=== 5 Data Pertama ===")
display(df.head())

# Mengetahui informasi struktur dataset (jumlah baris, kolom, tipe data, missing value)
print("\n=== Informasi Dataset ===")
df.info()

# Statistik deskriptif dari data numerik
print("\n=== Statistik Deskriptif ===")
display(df.describe())
```

Output

Lima data pertama (df.head()):

```
=== 5 Data Pertama ===
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	0
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	1
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	0
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	0

Informasi struktur dataset (df.info()):

```
=== Informasi Dataset ===
<class 'pandas.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Age             918 non-null    int64
 1   Sex             918 non-null    str
 2   ChestPainType   918 non-null    str
 3   RestingBP       918 non-null    int64
 4   Cholesterol     918 non-null    int64
 5   FastingBS       918 non-null    int64
 6   RestingECG      918 non-null    str
 7   MaxHR           918 non-null    int64
 8   ExerciseAngina  918 non-null    str
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    str
 11  HeartDisease    918 non-null    int64
dtypes: float64(1), int64(6), str(5)
memory usage: 86.2 KB
```

Statistik deskriptif (df.describe()):

```
=== Statistik Deskriptif ===
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease
count	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000
mean	53.510893	132.396514	198.799564	0.233115	136.809368	0.887364	0.553377
std	9.432617	18.514154	109.384145	0.423046	25.460334	1.066570	0.497414
min	28.000000	0.000000	0.000000	0.000000	60.000000	-2.600000	0.000000
25%	47.000000	120.000000	173.250000	0.000000	120.000000	0.000000	0.000000
50%	54.000000	130.000000	223.000000	0.000000	138.000000	0.600000	1.000000
75%	60.000000	140.000000	267.000000	0.000000	156.000000	1.500000	1.000000
max	77.000000	200.000000	603.000000	1.000000	202.000000	6.200000	1.000000

Penjelasan

pd.read_csv() memuat berkas CSV ke dalam DataFrame df. df.head() menampilkan lima baris pertama untuk memastikan data terbaca benar. df.info() menunjukkan bahwa dataset memiliki 918 baris dan 12 kolom (6 numerik bertipe int64/float64 dan 5 kategorikal bertipe string) dengan jumlah non-null 918 pada setiap kolom, sehingga tidak ada missing value. df.describe() merangkum statistik fitur numerik. Rata-rata HeartDisease 0,553 berarti sekitar 55,3% pasien (±508 dari 918) berlabel positif.

Catatan kualitas data: nilai minimum RestingBP dan Cholesterol adalah 0, padahal tekanan darah dan kolesterol bernilai 0 tidak masuk akal secara fisiologis sehingga kemungkinan merupakan

data yang hilang yang dikodekan sebagai 0. Nilai minimum Oldpeak juga negatif (−2,6). Pada notebook ini nilai-nilai tersebut tidak ditangani secara khusus sehingga dapat memengaruhi hasil model.

3.4 Cell 3 – EDA: Visualisasi Target dan Korelasi

Kode

[13]:
```
# NIM: 202431149
# Visualisasi distribusi target (HeartDisease)
plt.figure(figsize=(6, 4))
sns.countplot(x='HeartDisease', data=df, palette='Set2')
plt.title('Distribusi Target (Heart Disease)')
plt.xlabel('Heart Disease (0 = Tidak, 1 = Ya)')
plt.ylabel('Jumlah')
plt.show()

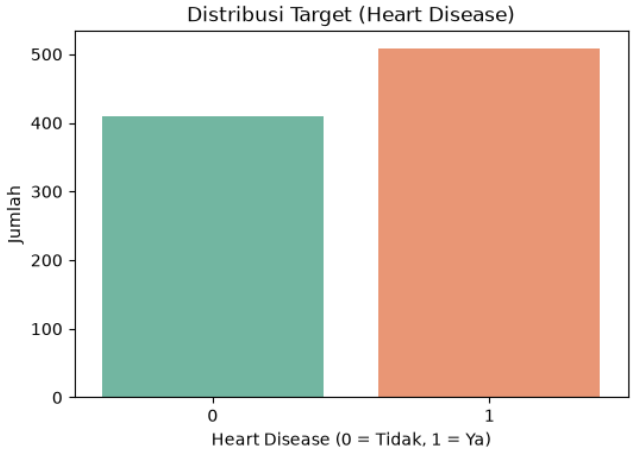
# Visualisasi Korelasi antar Fitur Numerik menggunakan Heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(df.select_dtypes(include=['int64', 'float64']).corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Matriks Korelasi Fitur Numerik')
plt.show()
```

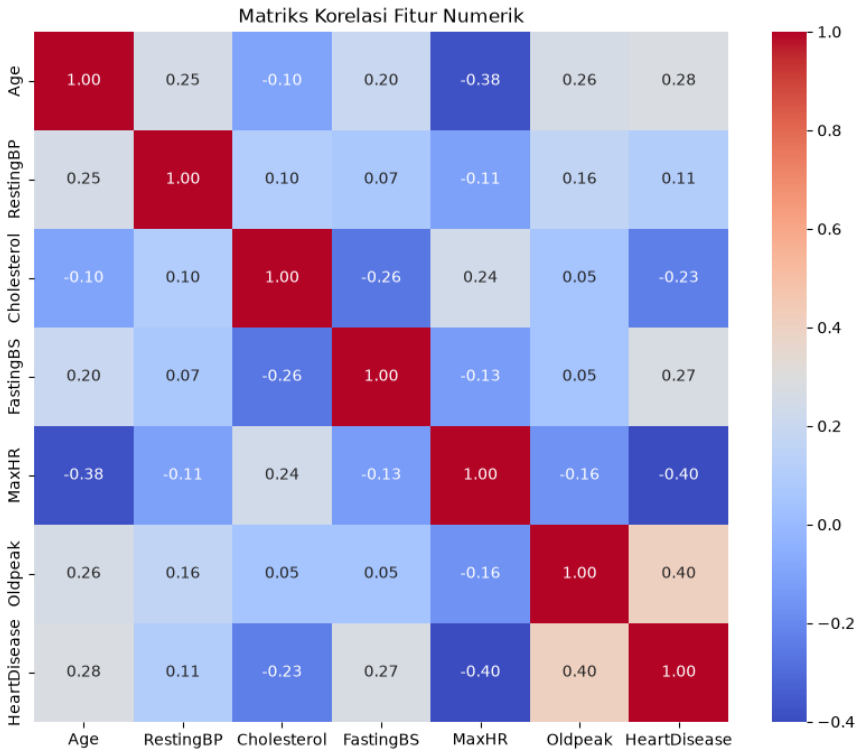
```
C:\Users\Ben S.A\AppData\Local\Temp\ipykernel_4252\447653770.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the s
ame effect.

  sns.countplot(x='HeartDisease', data=df, palette='Set2')
```

Output





Penjelasan

Bagian pertama membuat countplot untuk melihat distribusi kelas target. Terlihat kelas 1 (penyakit jantung) lebih banyak daripada kelas 0 namun selisihnya tidak besar (sekitar 508 berbanding 410), sehingga dataset dapat dianggap relatif seimbang dan akurasi masih layak dipakai sebagai metrik utama. Bagian kedua menampilkan heatmap korelasi Pearson untuk semua kolom numerik. Korelasi terkuat dengan target adalah Oldpeak (+0,40) dan MaxHR (−0,40), diikuti Age (+0,28), FastingBS (+0,27), dan Cholesterol (−0,23). Artinya, pasien dengan depresi segmen ST (Oldpeak) lebih tinggi dan detak jantung maksimum lebih rendah cenderung berpenyakit jantung. Korelasi Cholesterol yang negatif terhadap target tampak berlawanan dengan dugaan medis dan kemungkinan dipengaruhi oleh banyaknya nilai kolesterol 0 pada data. Seaborn menampilkan FutureWarning terkait parameter palette tanpa hue; peringatan ini tidak memengaruhi hasil.

3.5 Cell 4 – Preprocessing dan Split Data

Kode

PreProcessing & Split Data

```
[14]: # NIM: 202431149
# Mengubah fitur kategorikal (string) menjadi numerik menggunakan One-Hot Encoding
df = pd.get_dummies(df, drop_first=True)

# Memisahkan Fitur (x) dan Target (y)
x = df.drop(['HeartDisease'], axis=1) # Variabel independen
y = df['HeartDisease']                # Variabel dependen / target

print("=== Fitur Setelah Encoding ===")
display(x.head())

# Membagi data menjadi Data Latih (Train) 75% dan Data Uji (Test) 25%
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25, random_state=42)

print(f'Ukuran x_train: {x_train.shape}')
print(f'Ukuran x_test : {x_test.shape}')
```

Output

Ukuran data setelah split:

```
=== Fitur Setelah Encoding ===
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	Sex_M	ChestPainType_ATA	ChestPainType_NAP	ChestPainType_TA	RestingECG_Normal	RestingECG_ST	Ex
0	40	140	289	0	172	0.0	True	True	False	False	True	False	
1	49	160	180	0	156	1.0	False	False	True	False	True	False	
2	37	130	283	0	98	0.0	True	True	False	False	False	True	
3	48	138	214	0	108	1.5	False	False	False	False	True	False	
4	54	150	195	0	122	0.0	True	False	True	False	True	False	

```
Ukuran x_train: (688, 15)
Ukuran x_test : (230, 15)
```

Fitur hasil encoding (x.head()) terdiri atas 15 kolom: Age, RestingBP, Cholesterol, FastingBS, MaxHR, Oldpeak, Sex_M, ChestPainType_ATA, ChestPainType_NAP, ChestPainType_TA, RestingECG_Normal, RestingECG_ST, ExerciseAngina_Y, ST_Slope_Flat, dan ST_Slope_Up.

Penjelasan

pd.get_dummies(df, drop_first=True) melakukan one-hot encoding pada lima kolom kategorikal. Dengan drop_first=True satu kategori tiap kolom dihilangkan sebagai pembanding (ASY, LVH, dan Down), sehingga jumlah fitur menjadi 15. Selanjutnya dataset dipisah menjadi x (seluruh kolom kecuali HeartDisease) dan y (HeartDisease). train_test_split membagi data dengan test_size = 0,25 dan random_state = 42 sehingga diperoleh 688 data latih (75%) dan 230 data uji (25%); random_state menjamin pembagian yang sama setiap kali dijalankan.

3.6 Cell 5 – Standardisasi Fitur

Kode

KNN

```
[15]: # NIM: 202431149
# Melakukan Standard Scaler agar rentang nilai fitur seragam (sangat penting untuk KNN)
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(x_train)

x_train_scaled = scaler.transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Penjelasan

StandardScaler dibuat dan di-fit hanya pada x_train sehingga rata-rata dan simpangan baku dihitung dari data latih saja. Hasil transformasi disimpan sebagai x_train_scaled dan x_test_scaled. Langkah ini penting untuk KNN karena jarak Euclidean akan didominasi fitur berskala besar seperti Cholesterol (hingga 603) jika tidak diseragamkan. Cell ini tidak menghasilkan output.

3.7 Cell 6 – Pemodelan KNN (K = 7)

Kode

```
# NIM: 202431149
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Inisialisasi dan training model KNN (Menggunakan K=7)
K_TARGET = 7
knn = KNeighborsClassifier(n_neighbors=K_TARGET)
knn.fit(x_train_scaled, y_train)

# Prediksi dan Evaluasi KNN
y_pred_knn = knn.predict(x_test_scaled)
akurasi_knn = accuracy_score(y_test, y_pred_knn)

print(f"=== EVALUASI MODEL KNN (K={K_TARGET}) ===")
print(f"Akurasi KNN: {akurasi_knn:.4f}")
print("\nClassification Report KNN:\n", classification_report(y_test, y_pred_knn))

# Visualisasi Confusion Matrix KNN
plt.figure(figsize=(5, 4))
sns.heatmap(confusion_matrix(y_test, y_pred_knn), annot=True, fmt='d', cmap='Greens')
plt.title(f'Confusion Matrix - KNN (K={K_TARGET})')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```

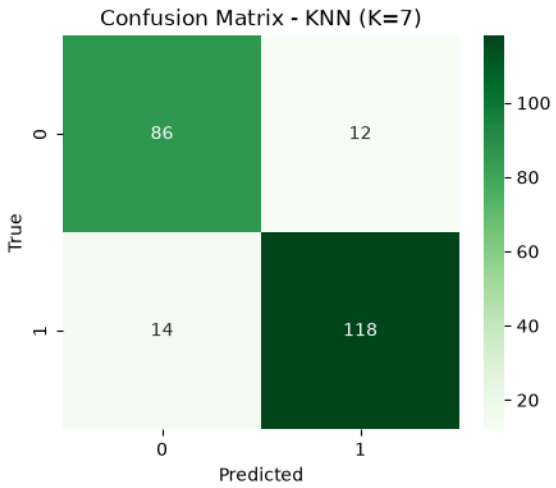
Output

```
=== EVALUASI MODEL KNN (K=7) ===
Akurasi KNN: 0.8870

Classification Report KNN:
              precision    recall  f1-score   support

           0       0.86      0.88      0.87        98
           1       0.91      0.89      0.90       132

    accuracy                           0.89       230
   macro avg       0.88      0.89      0.88       230
weighted avg       0.89      0.89      0.89       230
```



Penjelasan

Model KNeighborsClassifier dilatih dengan n_neighbors = 7 pada data yang sudah distandardisasi, lalu dipakai memprediksi data uji. Akurasi yang diperoleh adalah 0,8870 (88,70%). Confusion matrix menunjukkan 86 TN, 12 FP, 14 FN, dan 118 TP, yaitu 204 dari 230 data uji terklasifikasi benar. Untuk kelas 1 (penyakit jantung) precision 0,91, recall 0,89, dan F1-score 0,90; untuk kelas 0 precision 0,86, recall 0,88, dan F1-score 0,87. Kinerja kedua kelas berimbang tanpa bias yang mencolok terhadap salah satu kelas.

3.8 Cell 7 – Pencarian Nilai K Optimal

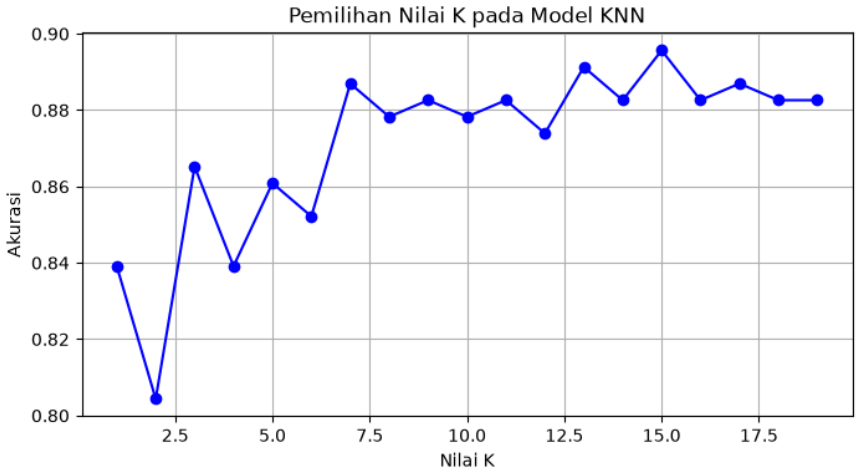
Kode

```
# NIM: 202431149
# Mencari nilai K optimal menggunakan looping
k_values = range(1, 20)
accuracies = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(x_train_scaled, y_train)
    y_pred_k = model.predict(x_test_scaled)
    accuracies.append(accuracy_score(y_test, y_pred_k))

# Plot Pemilihan Nilai K
plt.figure(figsize=(8, 4))
plt.plot(k_values, accuracies, marker='o', color='b')
plt.xlabel("Nilai K")
plt.ylabel("Akurasi")
plt.title("Pemilihan Nilai K pada Model KNN")
plt.grid(True)
plt.show()
```

Output



Penjelasan

Perulangan mencoba K dari 1 sampai 19, melatih KNN pada setiap nilai K, dan mencatat akurasi pada data uji. Grafik menunjukkan akurasi rendah dan tidak stabil untuk K kecil (K = 2 sekitar 0,80; K = 1 sekitar 0,84) karena model sensitif terhadap noise. Mulai K = 7 akurasi naik dan relatif stabil di kisaran 0,87–0,90. Nilai tertinggi terlihat pada K = 15 (sekitar 0,896), diikuti K = 13 (sekitar 0,891). K = 7 (0,887) dipilih pada Cell 6 sebagai titik awal plateau akurasi yang cukup sederhana. Perlu dicatat bahwa pencarian K di sini menggunakan data uji sehingga hasilnya cenderung optimistis; cara yang lebih tepat adalah cross-validation pada data latih.

3.9 Cell 8 – Pemodelan Decision Tree

Kode

DECISION TREE

```
[19]: # NIM: 202431149
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree

# Inisialisasi dan training model Decision Tree (menggunakan data asli tanpa scaling)
dt_classifier = DecisionTreeClassifier(random_state=42)
dt_classifier.fit(x_train, y_train)

# Prediksi dan Evaluasi Decision Tree
y_pred_dt = dt_classifier.predict(x_test)
akurasi_dt = accuracy_score(y_test, y_pred_dt)

print("=== EVALUASI MODEL DECISION TREE ===")
print(f"Akurasi Decision Tree: {akurasi_dt:.4f}")
print("\nClassification Report Decision Tree:\n", classification_report(y_test, y_pred_dt))
print(f"Kedalaman Pohon (Tree Depth): {dt_classifier.get_depth()}")

# Visualisasi Pohon Keputusan (Decision Tree)
plt.figure(figsize=(16, 8))
tree.plot_tree(dt_classifier, filled=True, feature_names=x.columns, class_names=['Normal', 'Heart Disease'], fontsize=8)
plt.title("Visualisasi Decision Tree")
plt.show()

# Visualisasi Confusion Matrix Decision Tree
plt.figure(figsize=(5, 4))
sns.heatmap(confusion_matrix(y_test, y_pred_dt), annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Decision Tree')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```

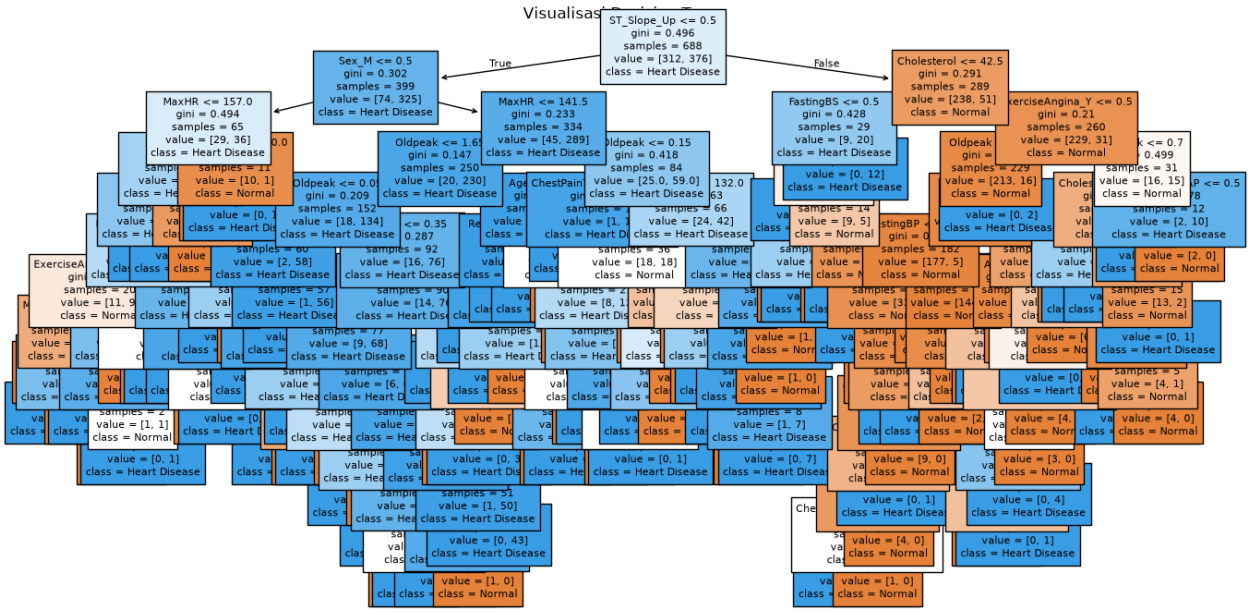
Output

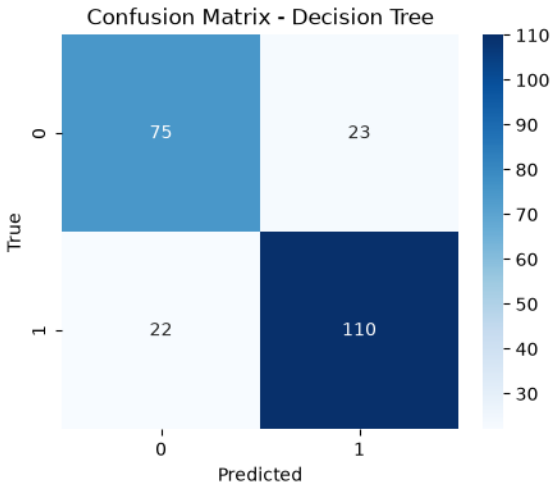
```
=== EVALUASI MODEL DECISION TREE ===
Akurasi Decision Tree: 0.8043

Classification Report Decision Tree:
              precision    recall  f1-score   support

           0       0.77      0.77      0.77        98
           1       0.83      0.83      0.83       132

    accuracy                           0.80       230
   macro avg       0.80      0.80      0.80       230
weighted avg       0.80      0.80      0.80       230
```





Penjelasan

DecisionTreeClassifier dilatih dengan random_state = 42 tanpa pembatasan kedalaman pada data asli (tanpa scaling, karena pohon tidak bergantung pada jarak). Akurasi yang diperoleh 0,8043 (80,43%) dengan kedalaman pohon 13. Confusion matrix menunjukkan 75 TN, 23 FP, 22 FN, dan 110 TP (185 prediksi benar). Precision, recall, dan F1-score kelas 0 masing-masing 0,77, sedangkan kelas 1 masing-masing 0,83. Fungsi tree.plot_tree menggambarkan pohon keputusan dengan simpul yang diwarnai menurut kelas dominan. Kedalaman 13 menandakan pohon tumbuh sangat rinci sehingga rentan overfitting terhadap data latih (Alkattan et al., 2025; Enihe et al., 2025).

3.10 Perbandingan Model dan Analisis

Metrik	KNN (K = 7)	Decision Tree
Akurasi	0,8870	0,8043
Precision kelas 0 / 1	0,86 / 0,91	0,77 / 0,83
Recall kelas 0 / 1	0,88 / 0,89	0,77 / 0,83
F1-score kelas 0 / 1	0,87 / 0,90	0,77 / 0,83
TN / FP / FN / TP	86 / 12 / 14 / 118	75 / 23 / 22 / 110
Prediksi benar (dari 230)	204	185
Perlu scaling	Ya	Tidak

Tabel 3.3 Perbandingan kinerja KNN dan Decision Tree pada data uji

Analisis. KNN unggul dengan akurasi 88,70% dibanding 80,43% pada Decision Tree, selisih sekitar 8,3 poin persentase atau 19 data uji lebih banyak yang terklasifikasi benar. Keunggulan ini konsisten pada seluruh metrik. Pada konteks medis, jumlah negatif palsu (pasien sakit yang diprediksi sehat) lebih penting: KNN menghasilkan 14 FN sedangkan Decision Tree 22 FN. Ada beberapa faktor yang menjelaskan perbedaan ini. Pertama, KNN dilatih pada data yang distandardisasi sehingga semua fitur berkontribusi seimbang terhadap jarak. Kedua, Decision Tree dibiarkan tumbuh sampai kedalaman 13 tanpa pruning sehingga menghafal data latih dan kurang mampu menggeneralisasi, sesuai temuan Alkattan et al. (2025) dan Enihe et al. (2025) mengenai overfitting pada pohon yang dalam. Ketiga, dataset berukuran kecil (688 data latih) sehingga hasil dapat berubah bila pembagian data berbeda.

Keterbatasan. (1) Nilai 0 pada RestingBP dan Cholesterol tidak ditangani; (2) nilai K dipilih memakai data uji; (3) evaluasi hanya memakai satu kali pembagian data sehingga belum diuji dengan cross-validation; (4) Decision Tree belum dituning (max_depth atau ccp_alpha). Keterbatasan ini menjadi dasar saran pada Bab IV.

BAB IV
PENUTUP

4.1 Kesimpulan

1. Dataset Heart Disease berisi 918 data dengan 11 fitur dan 1 target tanpa missing value; kelas target relatif seimbang (±55% positif). Oldpeak dan MaxHR memiliki korelasi terkuat dengan target (|r| = 0,40).
2. Preprocessing berupa one-hot encoding menghasilkan 15 fitur, split 75:25 menghasilkan 688 data latih dan 230 data uji, dan StandardScaler yang di-fit pada data latih digunakan untuk KNN.
3. KNN (K = 7) menghasilkan akurasi 0,8870, sedangkan Decision Tree menghasilkan akurasi 0,8043 dengan kedalaman 13. KNN lebih baik pada seluruh metrik dan memiliki negatif palsu lebih sedikit (14 berbanding 22).
4. Sesuai landasan teori, keunggulan KNN didukung oleh standardisasi fitur, sedangkan kinerja Decision Tree yang lebih rendah kemungkinan dipengaruhi overfitting akibat pohon yang tidak dipangkas.

4.2 Saran

1. Menangani nilai 0 pada RestingBP dan Cholesterol (misalnya dengan imputasi median) sebelum pemodelan.
2. Memilih K dan hyperparameter Decision Tree (max_depth, ccp_alpha) menggunakan cross-validation atau GridSearchCV, bukan data uji.
3. Menambah pembanding algoritma seperti Random Forest, SVM, atau Logistic Regression, dan menambah metrik seperti ROC-AUC.

DAFTAR PUSTAKA

Alkattan, H., Alhumaima, A. S., Mohmood, M. S., & AL-Thabhawee, G. D. M. (2025). Optimizing decision tree classifiers for healthcare predictions: A comparative analysis of model depth, pruning, and performance. Mesopotamian Journal of Artificial Intelligence in Healthcare, 3(1), 124–135. https://doi.org/10.58496/MJAIH/2025/013

Enihe, R. O., Prasad, R., Ogwueleka, F. N., & Abdullahi, F. B. (2025). The effect of imbalance data mitigation techniques on cardiovascular disease prediction. Journal of the Nigerian Society of Physical Sciences, 7, 2385. https://doi.org/10.46481/jnsps.2025.2385

Fatah, Z., & Aida, N. (2026). Application of the K-Nearest Neighbor (KNN) algorithm in data mining for heart disease prediction. COREAI: Jurnal Kecerdasan Buatan, Komputasi dan Teknologi Informasi, 7(1), 1–7. https://doi.org/10.33650/coreai.v7i1.13481

Harikiran, J., Lakshmi, Y. B., Kumar, A. P., & Verma, A. (2024). Predictive modelling for heart disease diagnosis: A comparative study of classifiers. EAI Endorsed Transactions on Pervasive Health and Technology, 10(1). https://doi.org/10.4108/eetpht.10.5518

Majumder, A. B., Gupta, S., Singh, D., Acharya, B., Gerogiannis, V. C., Kanavos, A., & Pintelas, P. (2023). Heart disease prediction using concatenated hybrid ensemble classifiers. Algorithms, 16(12), 538. https://doi.org/10.3390/a16120538

Mary, N., Khan, B., Asiri, A. A., Muhammad, F., Khan, S., Alqhtani, S., Mehdar, K. M., Halwani, H. T., Irfan, M., & Alshamrani, K. A. (2022). Heart disease risk prediction expending of classification algorithms. Computers, Materials & Continua, 73(3), 6595–6616. https://doi.org/10.32604/cmc.2022.032384

Pargent, F., Pfisterer, F., Thomas, J., & Bischl, B. (2022). Regularized target encoding outperforms traditional methods in supervised machine learning with high cardinality features. Computational Statistics, 37(5), 2671–2692. https://doi.org/10.1007/s00180-022-01207-6

Takcı, H. (2023). Performance-enhanced KNN algorithm-based heart disease prediction with the help of optimum parameters. Journal of the Faculty of Engineering and Architecture of Gazi University, 38(1), 451–460. https://doi.org/10.17341/gazimmfd.977127